

Chapter 11

Generics in Java

Generics in Java allow classes, interfaces, and methods to work with different data types while providing compile-time type safety. They use type parameters such as T, E, K, and V to make code reusable without requiring unnecessary type casting.

- Reduce the need for explicit type casting.
- Allow classes, interfaces, and methods to work with different reference types.
- Help detect incompatible types during compilation.

Why Use Generics?

- Before generics, collections such as ArrayList stored elements as Object, so retrieving an element often required explicit type casting.
- This could lead to ClassCastException at runtime if an object was cast to an incompatible type.
- Generics allow you to specify the type of elements a collection can store, such as ArrayList<String>.
- The compiler checks type safety at compile time, reducing the need for explicit casting and helping detect type-related errors earlier.
- Generics also make code easier to read by clearly indicating the type of data a collection is intended to contain.

Types of Java Generics

1. Generic Class

A generic class is a class that can operate on objects of different types using a type parameter. Like C++, we use <> to specify parameter types in generic class creation. To create objects of a generic class, we use the following syntax:

```
// To create an instance of generic class  
BaseType <Type> obj = new BaseType <Type>()
```

```
// We use < > to specify Parameter type
```

```
class Test<T> {
```

```

T obj;

Test(T obj) {
    this.obj = obj;
}

public T getObject() { return this.obj; }
}

class Geeks {

    public static void main(String[] args)
    {
        // instance of Integer type
        Test<Integer> iObj = new Test<Integer>(15);
        System.out.println(iObj.getObject());

        // instance of String type
        Test<String> sObj
            = new Test<String>("GeeksForGeeks");
        System.out.println(sObj.getObject());
    }
}

```

Output

15

GeeksForGeeks

Note: In Parameter type, we can not use primitives like "int", "char" or "double". Use wrapper classes like Integer, Character, etc.

How Type Parameter T Behaves Like a Normal Type

In a generic class, the type parameter T behaves like a normal data type within the class. Once a specific type is provided while creating an object, the compiler replaces T with that type.

This means T can be used just like a regular type for:

- Declaring variables
- Method parameters
- Method return types

Limitations of Generics

1. Generics Work Only with Reference Types

When we declare an instance of a generic type, the type argument passed to the type parameter must be a reference type

2. Generic Types Differ Based on their Type Arguments

Generic types differ based on their type arguments, but this difference exists only at compile time. During compilation, Java removes generic type information through a process called type erasure, replacing type parameters with their bounds or Object.

Custom Generics in Java

You've likely used Java generics countless times: `ArrayList<String>`, `HashMap<Integer, Customer>`, `Optional<User>`. They provide wonderful type safety, catching errors at compile-time and eliminating messy casts. But have you ever thought about creating *your own* generic classes or interfaces

Custom generics work by introducing **type parameters** — placeholders for actual types that will be specified later. These parameters are declared within angle brackets (< >) right after your class or interface name.

Common conventions for type parameter names are single uppercase letters:

- T for Type
- E for Element (often used in collections)
- K for Key
- V for Value
- N for Number
- S, U, V etc. for subsequent types